Lecture Notes for

Neural Networks I

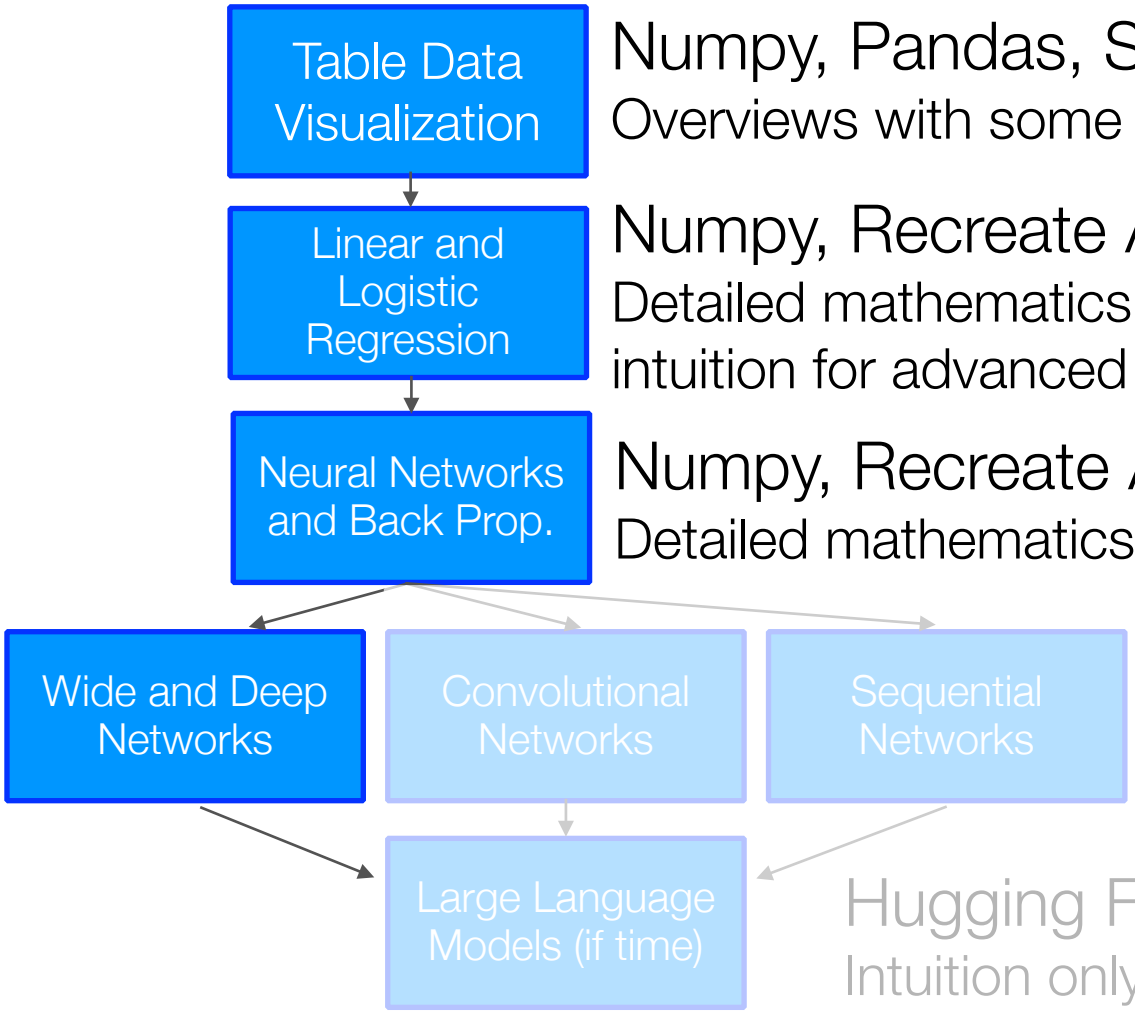


API Deep Dive

Lecture Agenda

- Logistics:
  - Grading update
- Agenda:
  - More Introduction to Torch
    - Tensors, modules
    - Deep APIs
  - Wide and Deep Networks
  - Town Hall

Class Overview, by topic



Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Hugging Face
Intuition only

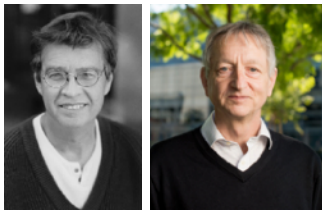
Last Time



1949, Hebb's Law
Close neuron fire together



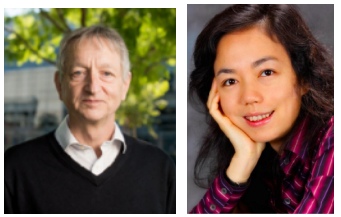
1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



2003, Vapnik
Kernel SVMs

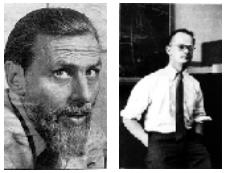


2012, Hinton, Fei-Fei Li
CNNs win ImageNet

1940 | 1960 | 1980 | 2000 | 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

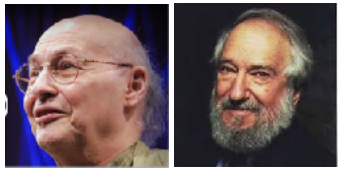
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



2015, Google
Tensorflow Open Sourced

TensorFlow

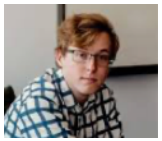




2015, Bengio, Schmidhuber
More Recurrent Nets

PyTorch
2017, PyTorch Beta

2018, Radford
GPT-1



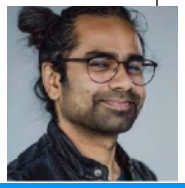
Hugging Face
2020, Models Hub

2018 - 2022
Various GPT Papers

Lots of Progress to Discuss
Various Papers

2018 | 2020 | 2022 | 2024 | 2026

Age of Deep Learning | Age of Large Language Models



2017, Vaswani
Attention is All You Need

2020, Brown
Few Shot Language Models



ChatGPT

2022, OpenAI
ChatGPT

Last Time: The computation graph

```
import torch
X = torch.tensor( … )
y = torch.tensor( … )
```

$$J(\mathbf{W}) = \frac{1}{N}\sum_{i}^{N}(y^{(i)} - \underbrace{(\mathbf{W}\cdot\mathbf{x}^{(i)} + \mathbf{b})}_{y_{pred}})^2$$

1. **Setup** Variables and computations

```
W = torch.randn( (1,n_features), requires_grad=True)
b = torch.randn( (1, 1),         requires_grad=True)

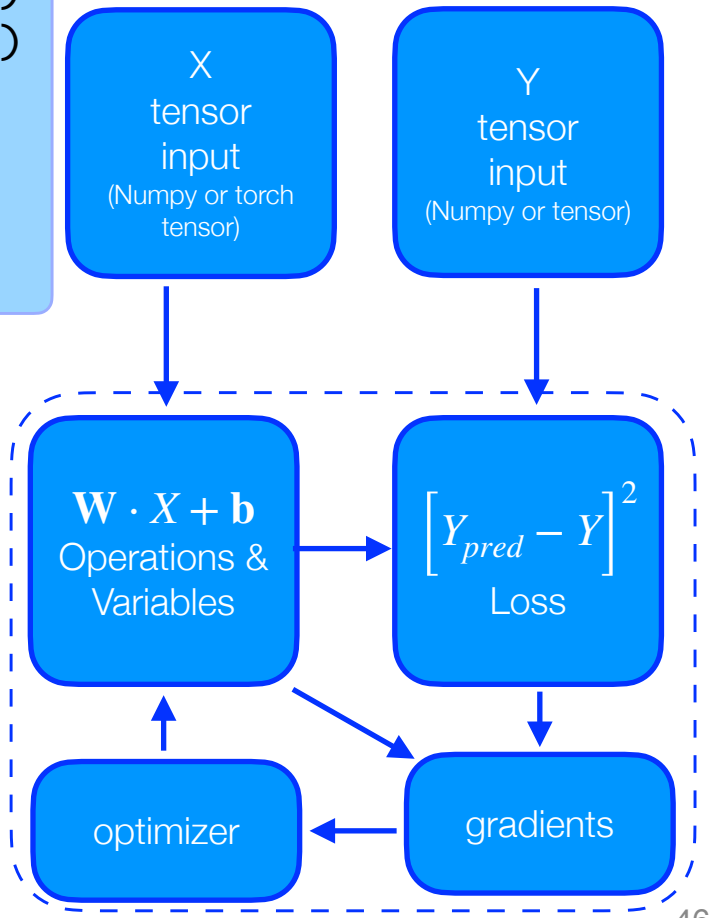
def feedforward(X,y):
    y_pred = torch.matmul(X, W) + b
    loss   = torch.mean((y-y_pred)**2)
    return loss
```

2. Add **optimization** operation to computation graph
   Adjusts variables (W, b) to minimize loss with auto differentiation

```
opt = torch.optim.SGD( [W, b], lr=0.01)
opt.zero_grad()

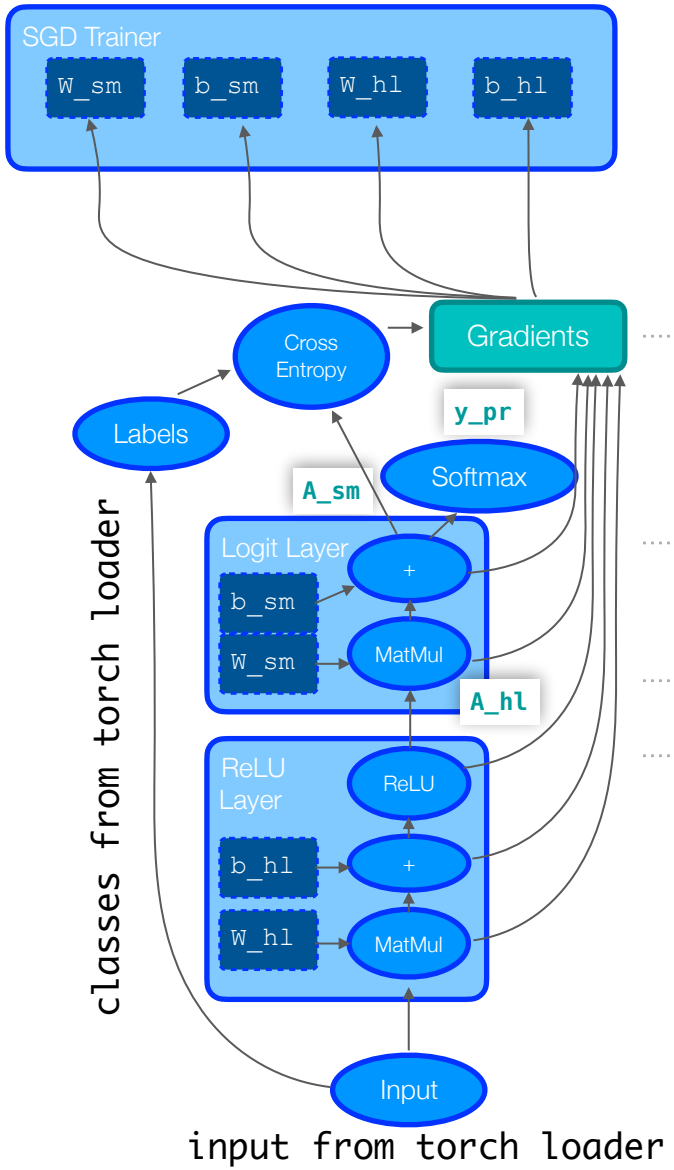
loss = feedforward(X, y)
loss.backward()
opt.step()
```

3. **Run graph operation** once, → one optimization update on all variables



http://www.datasciencecentral.com/profiles/blogs/google-open-source-tensorflow

Computation Graph, Two Layer Network




```
import torch
import torch.nn.functional as F
```

```
W_sm = torch.randn(…)
b_sm = torch.randn(…)
W_hl = torch.randn(…)
b_hl = torch.randn(…)
```

```
trainable_variables =
        [W_sm,b_sm,W_hl,b_hl]
```

```
def model_forward(Input):
  A_hl = torch.relu( torch.matmul(Input,W_hl) + b_hl )
  A_sm = torch.matmul(A_hl,W_sm) + b_sm
  return A_sm
```

maybe compile

```
  loss = F.cross_entropy(labels, A_sm)
  y_pr = torch.softmax(A_sm)
```

```
  opt = torch.optim.SGD(trainable_variables, lr=0.01)
```

```
for features, labels in train_data:
    logits   = model_forward(features)
    loss_val = F.cross_entropy(labels, logits)

    opt.zero_grad()
    loss_val.backward()
    opt.step()
```

torch data loader

Self Test

- The computation graph in PyTorch:
  - A. Must run one operation at a time
  - B. Can be “compiled” for efficiency
  - C. Has one input path (e.g., features) and one output path (e.g., predictions)
  - D. All of the Above

Using Deep Learning APIs



Programming Interfaces

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
model = nn.Sequential(
  nn.Linear(in_features=2, out_features=4),
  nn.ReLU(),
  nn.Linear(in_features=4, out_features=3) )
```

- **Sequential API**
  - great for simple, feed forward models

- **Classes (Inheritance)**
  - good for more advanced functionality
  - Can be used with Sequential API

```
class TwoLayerNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()

        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.relu   = nn.ReLU()
        self.layer2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out = self.layer1(x)
        out = self.relu(out)
        out = self.layer2(out)
        return out
```

```
model = TwoLayerNet(input_dim=2,
     hidden_dim=4, output_dim=3)
```

```
class TwoLayerNet(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.hidden_block = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU()  )
        self.classifier = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        features = self.hidden_block(x)
        logits = self.classifier(features)

        return logits
```

Training similar to our custom models

General Training Procedure

```
model = TwoLayerNet(input_dim=2, hidden_dim=4, output_dim=3)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

epochs = 5

for epoch in range(epochs):
    # Set model to training mode (activate dropout, etc.)
    model.train()

    optimizer.zero_grad()
    logits = model(X)
    loss = criterion(logits, Y)
    loss.backward()
    optimizer.step()
```

Accelerations

```
# Move to GPU or MPS
model = model.to("GPU")      model = model.to("MPS")


# Optimize Computation Graph
optimized_model = torch.compile(model)


# Use Hugging Face Accelerate API
from accelerate import Accelerator
model, optimizer, my_dataloader = accelerator.prepare(
                      model,optimizer,my_dataloader)
```

use specific hardware accelerations

merges operations to reduce framework overhead and VRAM

adapts loop code dynamically to the host hardware: CPUs, GPUs, or multi-GPU/TPU clusters

08. Pytorch Wide and Deep.ipynb

Demo

Variable	description
age:	continuous
workclass:	Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, ...
fnlwgt:	continuous.
education:	Bachelors, Some-college, 11th, HS-grad, Prof-school, ...
education-num:	continuous.
marital-status:	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, ...
occupation:	Tech-support, Craft-repair, Other-service, ...
relationship:	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
race:	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
sex:	Female, Male.
capital-gain:	continuous.
capital-loss:	continuous.
hours-per-week:	continuous.
native-country:	United-States, Cambodia, England, ...
income (target):	<50k, >=50k



```
Test Labels
<=50K    11360
>50K      3700

Training Labels
<=50K    22654
>50K      7508

workclass 7 unique values.
education 16 unique values.
marital_status 7 unique values.
occupation 14 unique values.
relationship 6 unique values.
race 5 unique values.
sex 2 unique values.
country 41 unique values.
```

Building a simple MLP with Torch

```
categorical_headers = ['workclass','education','marital_status',
                       'occupation','relationship','race','sex','country']

numeric_headers     = ['age', 'cap_gain', 'cap_loss','work_hrs_weekly','education_num']
```

Using the Data Loader (numeric only)



```
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset

class AdultDataset(Dataset):
    def __init__(self, df):
        self.X = torch.tensor( df[numeric_headers].to_numpy(), dtype=torch.float32)
        self.y = torch.tensor( df.income.to_numpy(),           dtype=torch.float32)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_loader = DataLoader( AdultDataset(df_train), batch_size=64, shuffle=True )
test_loader  = DataLoader( AdultDataset(df_test),  batch_size=64, shuffle=False)
```

Create **loader** from **numeric** features inherited from torch **Dataset**

```
feat, label = next(iter(train_loader))

print('\nThe feature shape for age is:')
print('Shape:',feat.shape)
print('Max:',feat.max())

print('\nThe labels range and shape:')
print(label.shape, label.min(),label.max())
```

```
The feature shape for age is:
Shape: torch.Size([64, 4])
Max: tensor(5178.)

The labels range and shape:
torch.Size([64, 1]) tensor(0.) tensor(1.)
```

Training with Simple Model

```
class IncomeClassifier(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 16), nn.ReLU(),
            nn.Linear(16, 8),         nn.ReLU(),
            nn.Linear(8, 1),          nn.Sigmoid()
        )

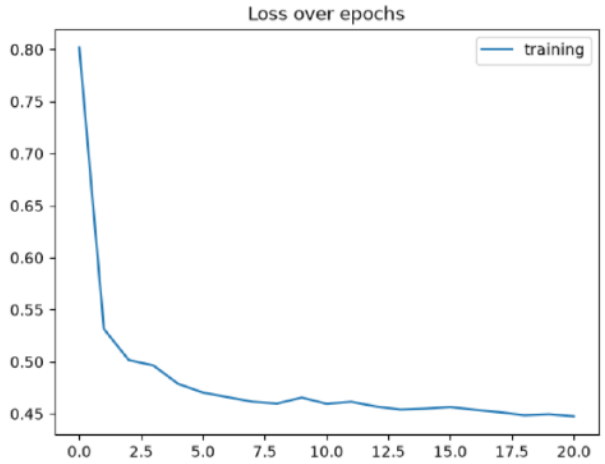
    def forward(self, x):
        return self.net(x)
```

```
IncomeClassifier(
  (net): Sequential(
    (0): Linear(in_features=4, out_features=16,
           bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=8,
           bias=True)
    (3): ReLU()
    (4): Linear(in_features=8, out_features=1,
           bias=True)
    (5): Sigmoid()
  )
)
```

```
model = IncomeClassifier(input_dim=4)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(EPOCHS):
    model.train()

    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()
        predictions = model(batch_X)
        loss = criterion(predictions, batch_y)
        loss.backward()
        optimizer.step()
```




```
Epoch 1  Loss: 0.8019 Test Accuracy: 0.7556
Epoch 11 Loss: 0.4596 Test Accuracy: 0.7882
Epoch 21 Loss: 0.4476 Test Accuracy: 0.7894
```

Accuracy of Test Data, Numeric Only: 78.94%

Categorical Feature Embeddings

- One hot encoded data can be made dense through a matrix multiplication, $\mathbf{a} = \mathbf{W}^{(emb)} \cdot \mathbf{x}_{OHE}$

Trainable Matrix, $\mathbf{W}^{(emb)}$, in Network

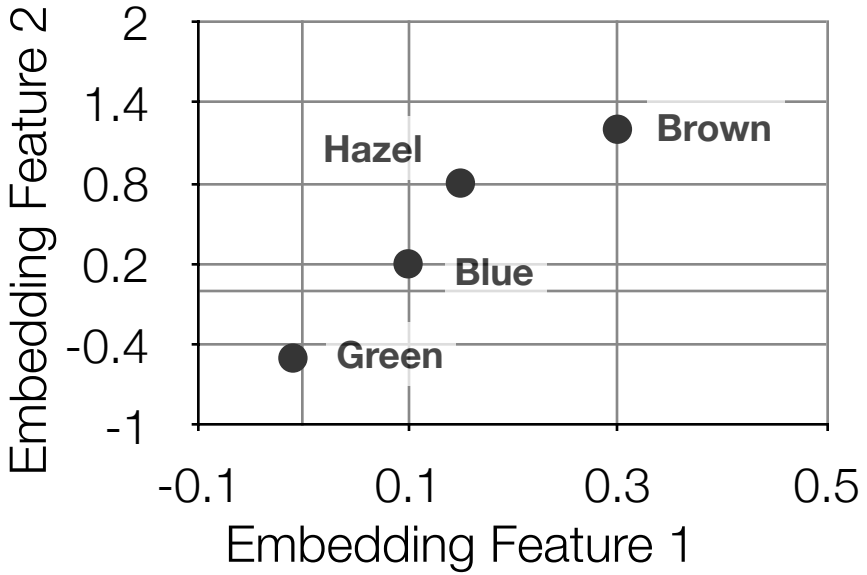
Reduced Dimensions

0.1	0.3	-0.01	0.15
0.2	1.2	-0.5	0.8

Num Categories

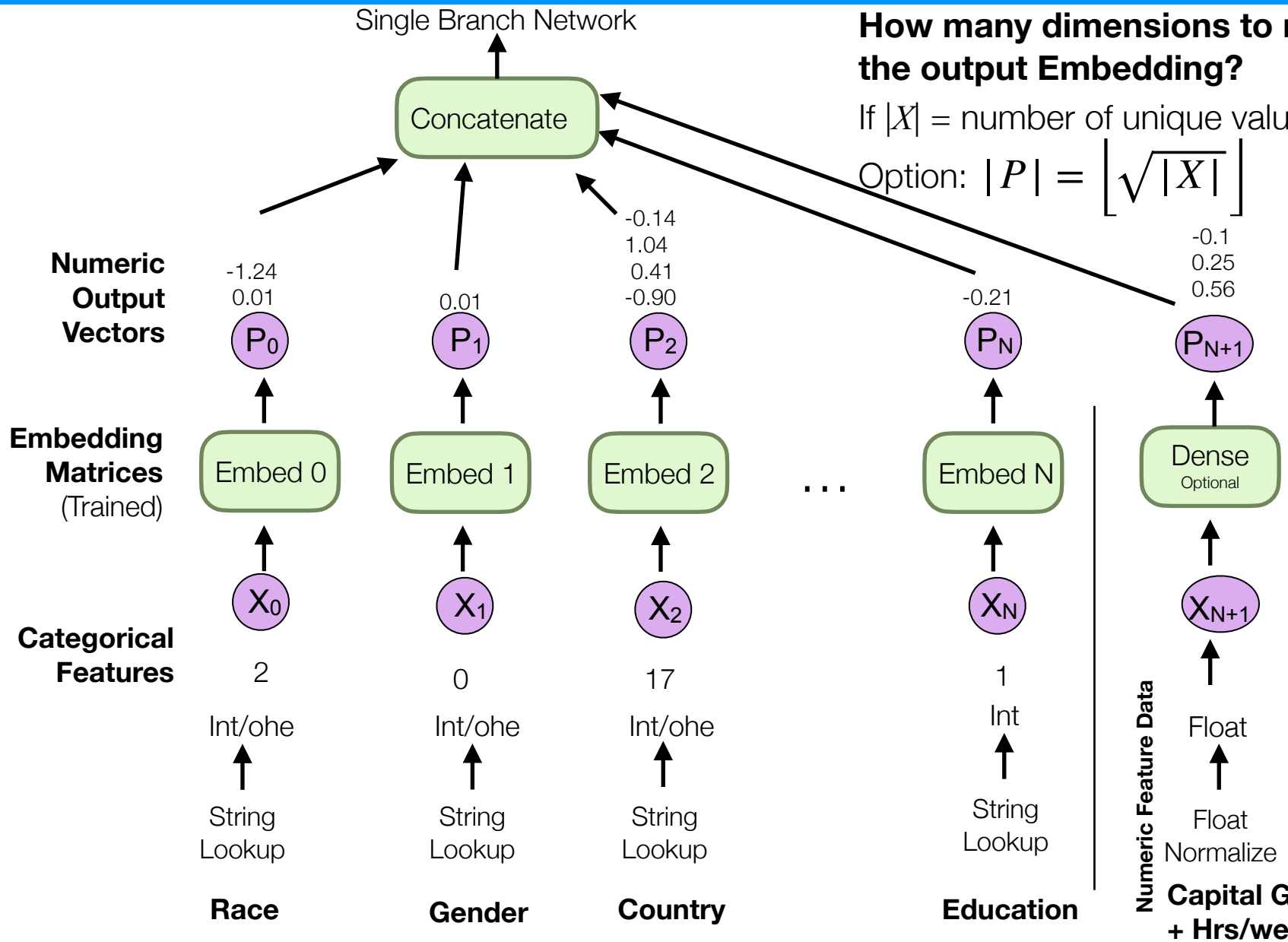
Eye Color

$$\cdot \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \begin{matrix} \text{Blue} \rightarrow 0 \\ \text{Brown} \rightarrow 1 \\ \text{Green} \rightarrow 2 \\ \text{Hazel} \rightarrow 3 \end{matrix}$$



Computationally: there is no need to one hot encode eye color, we can just use the integer to index into the **embedding matrix**

Using Embeddings in Torch



Loading Embed Data



```
embedding_sizes = {}
for col in categorical_headers:
    df_train[col+'_int'] = LabelEncoder().fit_transform(df_train[col])
    N = … get num_classes …
    emb_dim = int( np.sqrt(N + 1) )
    embedding_sizes[col+'_int'] = (N, emb_dim)


 class AdultEmbeddingDataset(Dataset):
     def __init__(self, df):
         cat_cols = [x+'_int' for x in categorical_headers]
         self.X_num = torch.tensor( df[numeric_headers].to_numpy(), …)
         self.X_cat = torch.tensor( df[cat_cols].to_numpy(),        …)
         self.y     = torch.tensor( df.income.to_numpy(),           …)

     def __getitem__(self, idx):
         return self.X_num[idx],   self.X_cat[idx],   self.y[idx]
```

How many dimensions to make the output Embedding?

If $|X|$ = number of unique values

Option: $|P| = \left\lfloor \sqrt{|X|} \right\rfloor$

```
embedding_sizes = {
     key (str)          value (tuple)
    ----------------   ----------------
    (workclass_int) : (7,  2)
    (education_int) : (16, 4)
   ...

    (country_int)   : (41, 6)
}
```

Building Embed Model

```
embedding_sizes = {
    key (str)          value (tuple)
    (workclass_int) : (7,  2)
    (education_int) : (16, 4)
    ...
    (country_int)   : (41, 6)
}
```

Create List of Embeddings

```
class IncomeEmbeddingClassifier(nn.Module):
    def __init__(self, embedding, num_numerical_cols):
        super().__init__()
        tmp = []
        total_emb_dim = 0
        for key, (vocab_size, embed_size) in embedding.items():
            tmp.append( nn.Embedding(vocab_size, embed_size) )
            total_emb_dim += embed_size
        self.embeddings = nn.ModuleList(tmp)

        total_input_dim = num_numerical_cols + total_emb_dim

        self.net = nn.Sequential(
                nn.Linear(total_input_dim, 32),  nn.ReLU(),
                nn.Linear(32, 16),               nn.ReLU(),
                nn.Linear(16, 1),                nn.Sigmoid()
            )
```

Create Network for Concatenated Vectors

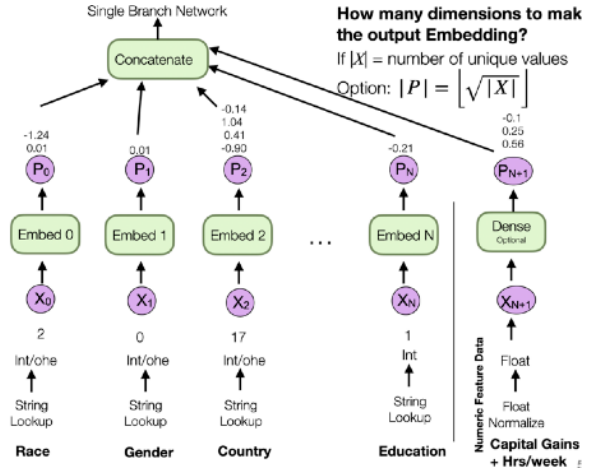
```
IncomeEmbeddingClassifier(
  (embeddings): ModuleList(
    (0): Embedding(7,  2)
    (1): Embedding(16, 4)
    (2): Embedding(7,  2)
    (3): Embedding(14, 3)
    (4): Embedding(6,  2)
    (5): Embedding(5,  2)
    (6): Embedding(2,  1)
    (7): Embedding(41, 6)
  )
  (net): Sequential(
    (0): Linear(in_features=26, out_fea
    (1): ReLU()
    (2): Linear(in_features=32, out_fea
    (3): ReLU()
    (4): Linear(in_features=16, out_fea
    (5): Sigmoid()
  )
)
```

21 embed + 5 numeric

```
    def forward(self, x_num (Dense matrix), x_cat (Integer matrix)):
        # Pass each categorical through respective embedding
        emb_outputs = [emb(x_cat[:, i])
                       for i, emb in enumerate(self.embeddings)]

        x_emb = torch.cat(emb_outputs, dim=1)
        x = torch.cat([x_num, x_emb], dim=1)

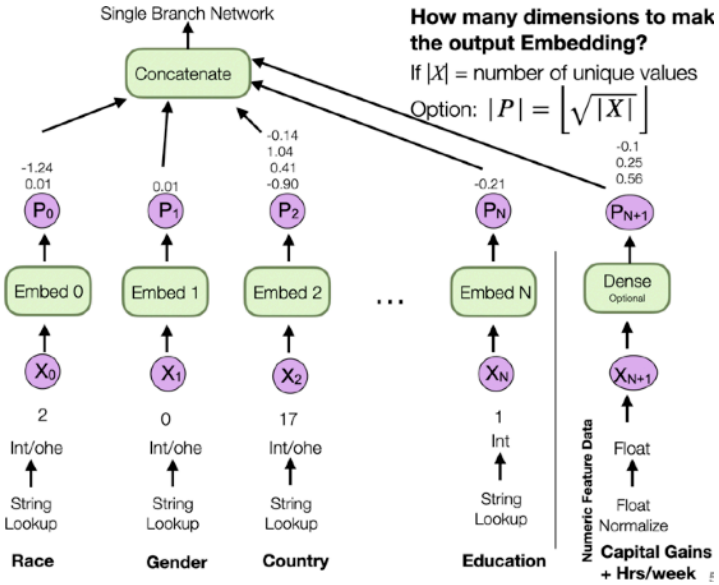
        return self.net(x)
```



Accuracy of Test Data, Numeric Only: 78.94%
Accuracy of Test Data, Embedded: 84.81%

Embedding +Numeric, Concatenation Review

```
(self.net): Sequential(
  (0): Linear(in_features=26, out_features=32, bias=True)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=16, bias=True)
  (3): ReLU()
  (4): Linear(in_features=16, out_features=1, bias=True)
  (5): Sigmoid()
```




```
(self.embeddings): ModuleList(
  (0): Embedding(7,  2)
  (1): Embedding(16, 4)
  (2): Embedding(7,  2)
  (3): Embedding(14, 3)
  (4): Embedding(6,  2)
  (5): Embedding(5,  2)
  (6): Embedding(2,  1)
  (7): Embedding(41, 6)
)
```

```
emb_outputs = []

for i, emb in enumerate(self.embeddings):
    val = emb( x_cat[:, i] )
    emb_outputs.append( val )

x_emb = torch.cat( emb_outputs, dim=1)
x = torch.cat([x_num, x_emb],   dim=1)

self.net(x)
```

Wide and Deep Networks



Wide and Deep

Wide & Deep Learning for Recommender Systems

Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, Hemal Shah

Google Inc.*

ABSTRACT

Generalized linear models with nonlinear feature transfor-

have never or rarely occurred in the past. Recommendations based on memorization are usually more topical and

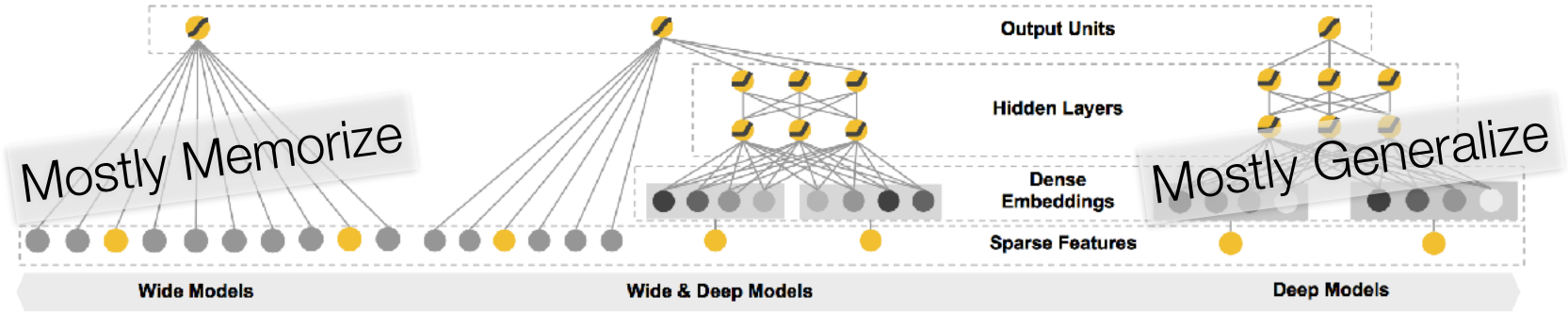
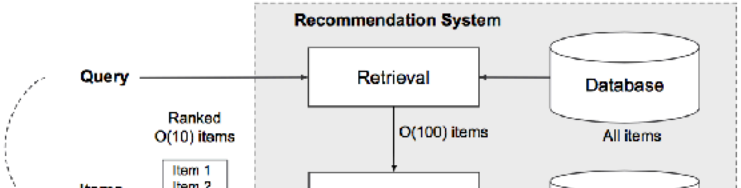


Figure 1: The spectrum of Wide & Deep models.

linear model with feature transformations for generic recommender systems with sparse inputs.

- The implementation and evaluation of the Wide & Deep recommender system productionized on Google



Why wide and deep?

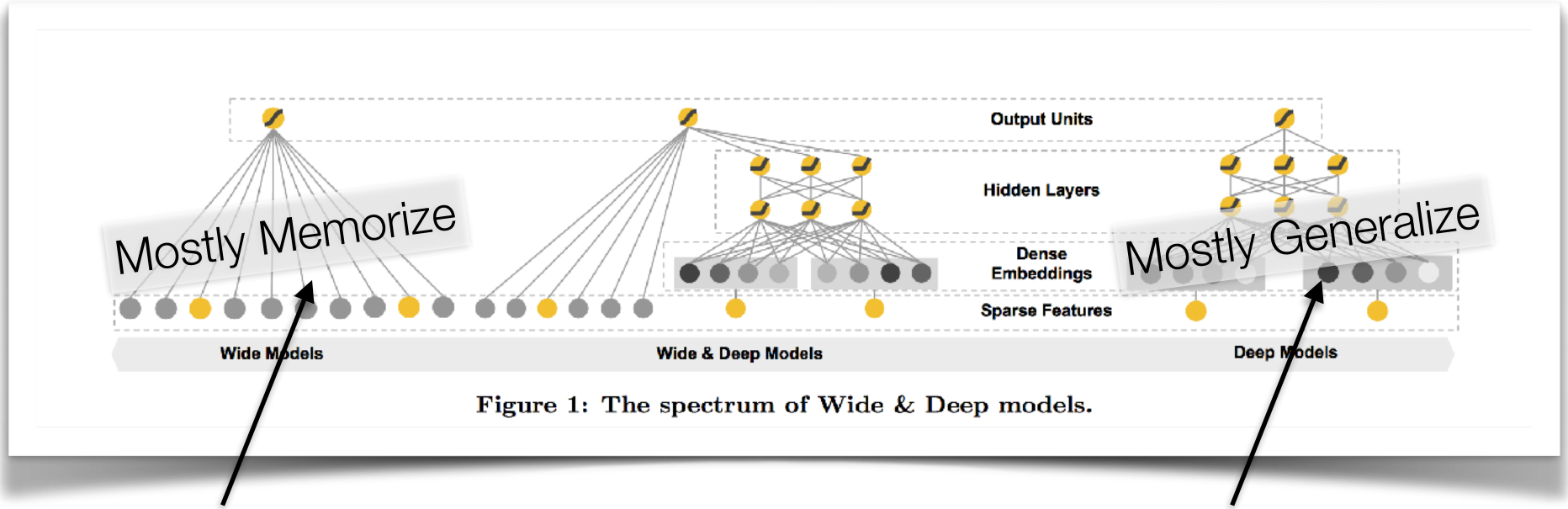


Figure 1: The spectrum of Wide & Deep models.

But why memorize?

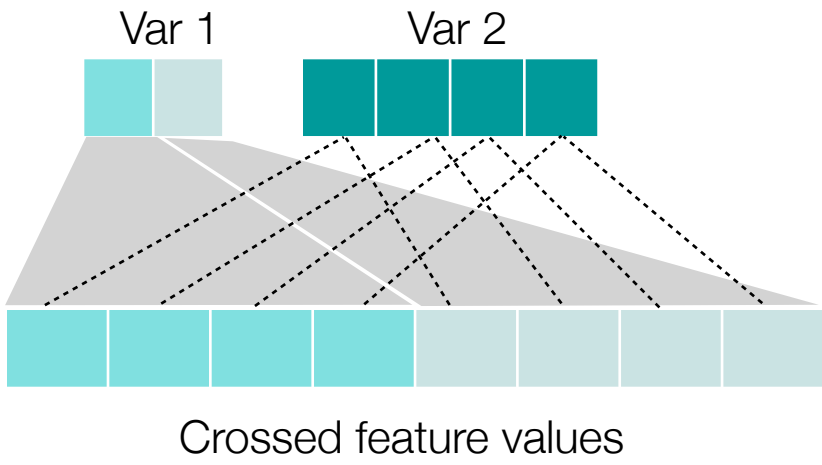
Obvious!

- Categorical values have combinations that repeat!
  - so memorizing these values is not necessarily a bad strategy
  - let's make memorizing easy on one network

Wide networks (Memorize?)

- Wide refers to the expansion of features set
- Crossed feature columns of categorical features

- Movie Rating
  - G
  - PG
  - PG-13
  - R
  - Else
- Movie Genre
  - Action
  - Drama
  - Comedy
  - Horror
  - Else

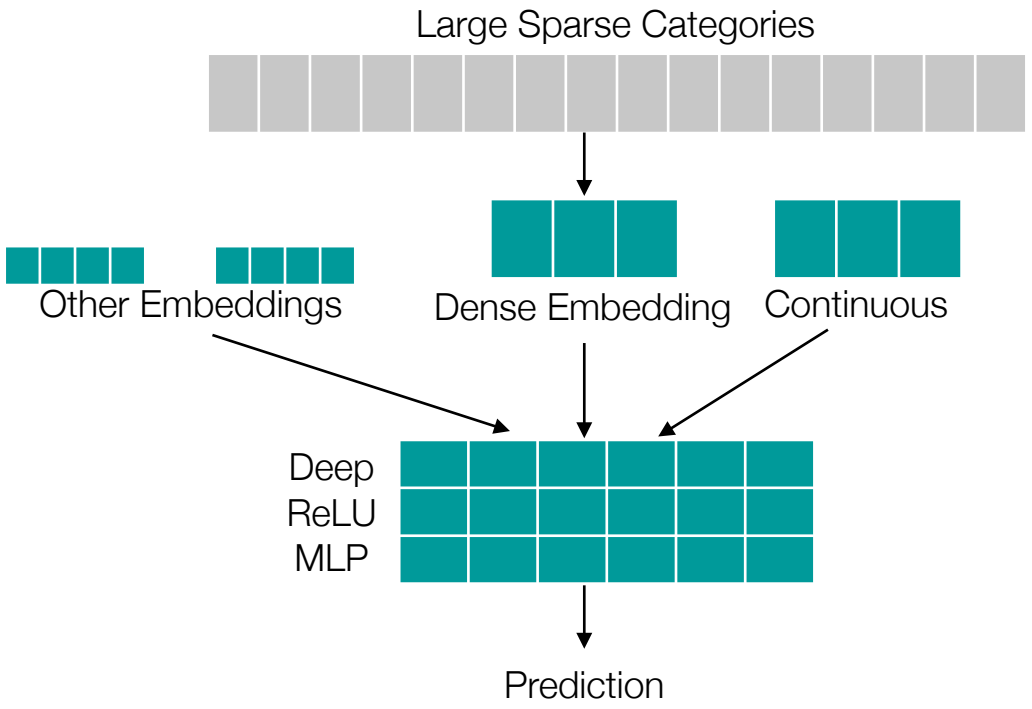


- Crossed feature “Rating-Genre”
  - G-Action, G-Drama, G-Comedy, G-Horror, G-else
  - PG-Action, PG-Drama, PG-Comedy, PG-Horror, G-else
  - and so on … one hot encoded

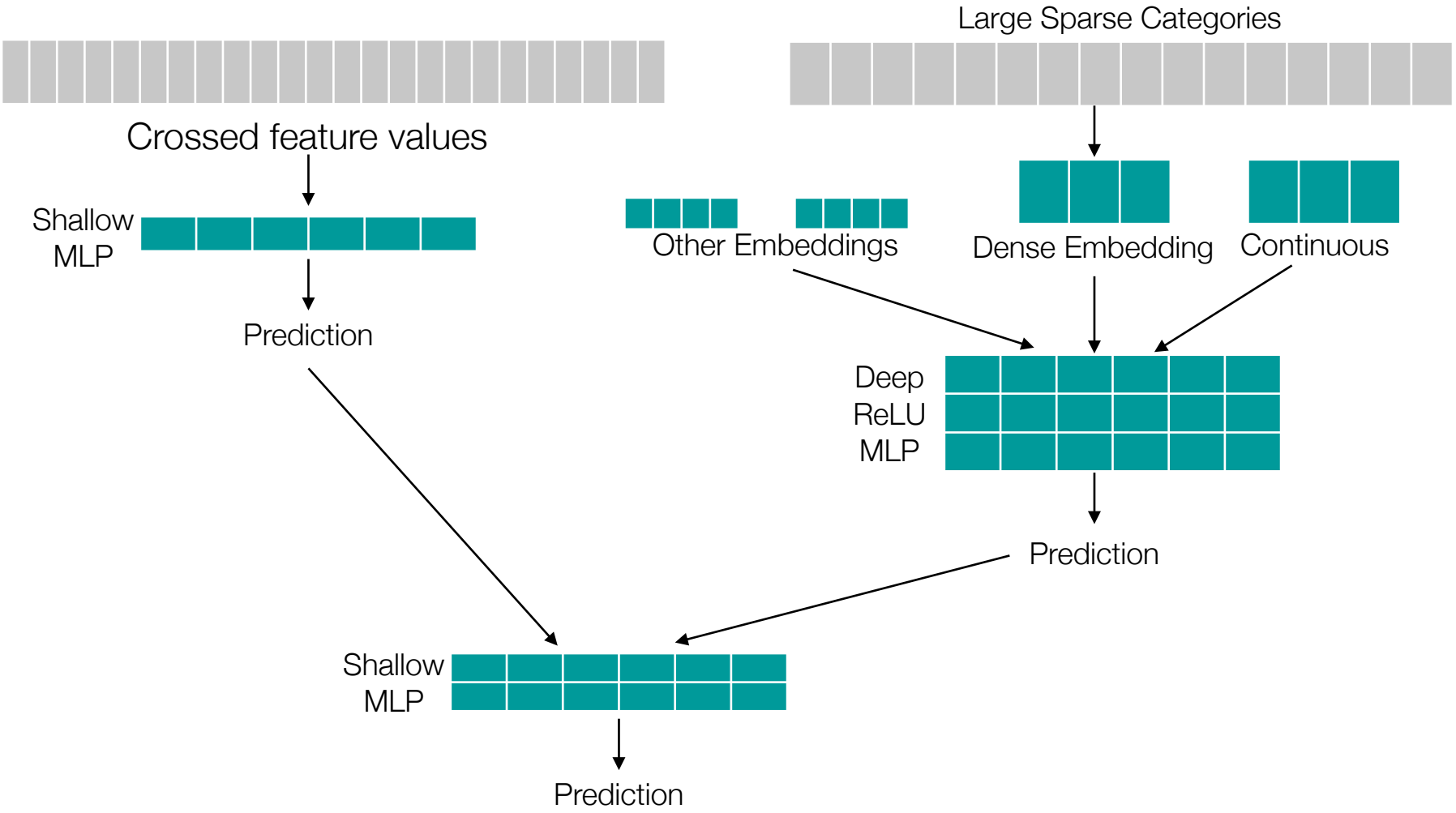
Deep Features: What we have already done!

- Deep refers to increasingly narrow hidden layers
- **Essentially the same as what we already did!**

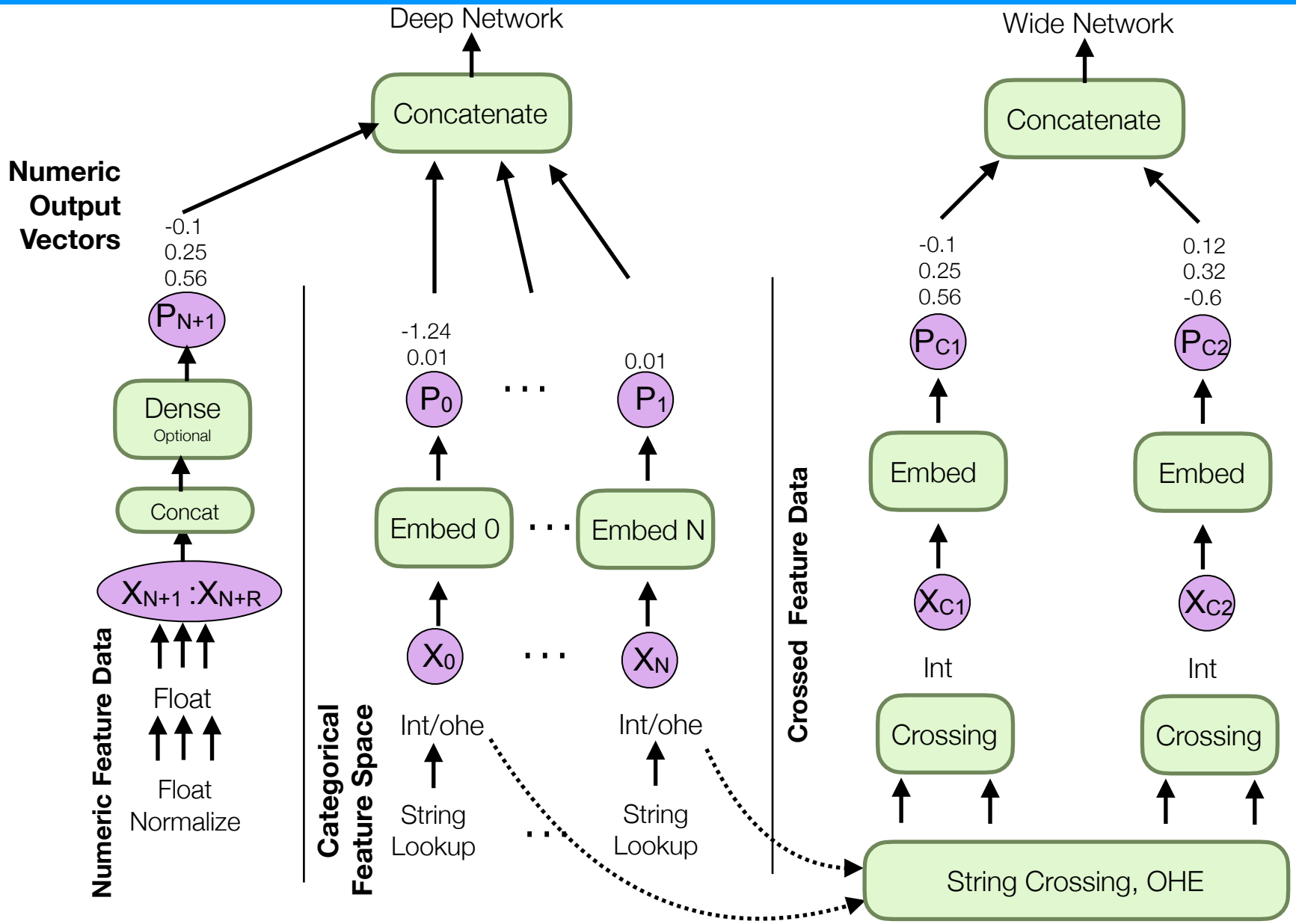
- Movie Actors
  - Armand Assante
  - Meryl Streep
  - Danny Trejo
  - Kevin Bacon
  - Audrey Hepburn
  - …
-



Combining Memorization and Generalization



Crossed Embeddings



Self Test

- For two categorical variables with possible categories of:
  - $V_1 \in \{A, B, C, D\}$
  - $V_2 \in \{\alpha, \beta, \gamma\}$
- how wide is $V_1 \times V_2$?

- A: Cannot determine
- B: 7
- C: 12
- D: 13

08. Pytorch Wide and Deep.ipynb

Demo

Variable	description
age:	continuous
workclass:	Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, ...
fnlwgt:	continuous.
education:	Bachelors, Some-college, 11th, HS-grad, Prof-school, ...
education-num:	continuous.
marital-status:	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, ...
occupation:	Tech-support, Craft-repair, Other-service, ...
relationship:	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
race:	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
sex:	Female, Male.
capital-gain:	continuous.
capital-loss:	continuous.
hours-per-week:	continuous.
native-country:	United-States, Cambodia, England, ...
income (target):	<50k, >=50k



```
Test Labels
<=50K    11360
>50K      3700

Training Labels
<=50K    22654
>50K      7508

workclass 7 unique values.
education 16 unique values.
marital_status 7 unique values.
occupation 14 unique values.
relationship 6 unique values.
race 5 unique values.
sex 2 unique values.
country 41 unique values.
```

Creating Crossed Embeddings …

Wide and Deep, Setup

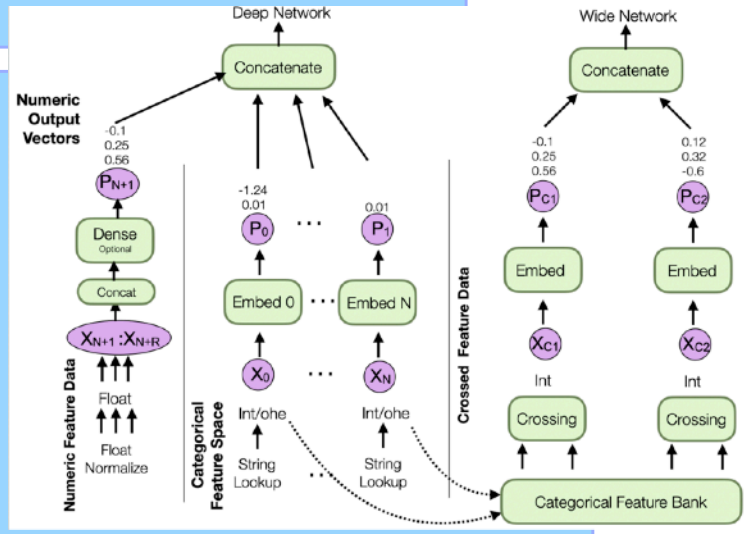
Demo

```
crossed_columns = []
crossing_sizes = {}
for feat1, feat2 in [('education','marital_status'), ('education','occupation') ]:
    col = f'{feat1}_x_{feat2}'
    df_train[col] = df_train[feat1].astype(str) + "_" + df_train[feat2].astype(str)

    crossed_columns.append(col)

    tmp = LabelEncoder()
    df_train[col+'_int'] = LabelEncoder().fit_transform(df_train[col])
    N = ... total classes ...
    emb_dim = int(np.sqrt(N + 1))
    crossing_sizes[col+'_int'] = (N, emb_dim)
```

feat1 (str)	feat2 (str)	crossing
Bachelors	married_af	Bachelors_ married_af
Some HS	single	Some HS_ single




```
class AdultCrossedDataset(Dataset):
    def __init__(self, df):
        # separate the categorical and feature crossings
        cat_cols = [x+'_int' for x in categorical_headers]
        crs_cols = [x+'_int' for x in crossed_columns]
        self.X_num = torch.tensor(df[numeric_headers], ...)
        self.X_cat = torch.tensor(df[cat_cols], ...)
        self.X_crs = torch.tensor(df[crs_cols], ...)
        self.y = torch.tensor(df.income, ...)

    def __getitem__(self, idx):
        return self.X_num[idx],   self.X_cat[idx],   self.X_crs[idx],   self.y[idx]

train_loader = DataLoader(AdultCrossedDataset(df_train), batch_size=64, shuffle=True)
```

Wide and Deep, Model



Demo

```
class IncomeWideDeepClassifier(nn.Module):
    def __init__(self, embedding, crossing, num_numerical_cols):
        ... same as code before ... and then ...
        self.embeddings = nn.ModuleList(tmp)
        self.deep_net   = nn.Sequential(… same as previous …)

        # feature crossing
        tmp = [],  total_crs_dim = 0
        for key, (vocab_size, embed_size) in crossing.items():
            tmp.append( nn.Embedding(vocab_size, embed_size) )
            total_crs_dim += embed_size

        self.crossings = nn.ModuleList(tmp)

        self.wide_net = nn.Sequential( nn.Linear(total_crs_dim, 8) )
        self.classify_net = nn.Sequential( nn.Linear(16, 1),
                                           nn.Sigmoid()     )

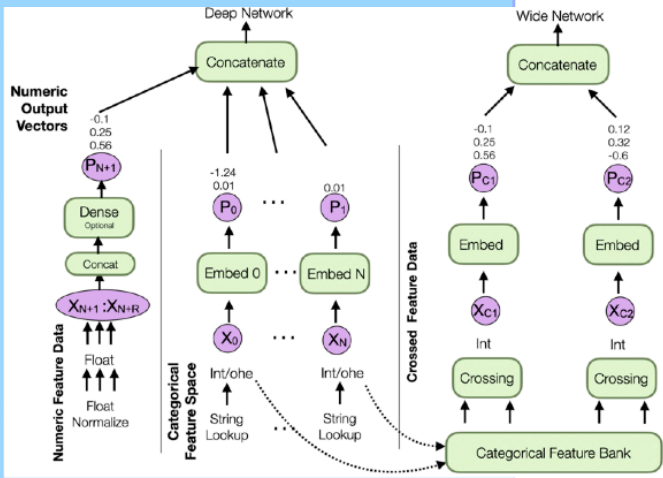
    def forward(self, x_num, x_cat, x_crs):
        emb_outputs = [emb(x_cat[:, i]) for i, emb in enumerate(self.embeddings)]

        x_emb = torch.cat(emb_outputs, dim=1)
        x = torch.cat([x_num, x_emb], dim=1)
        dx = self.deep_net(x)

        crs_outputs = [emb(x_crs[:, i]) for i, emb in enumerate(self.crossings)]
        x_emb_crs = torch.cat(crs_outputs, dim=1)
        cx = self.wide_net(x_emb_crs)

        x = torch.cat([dx, cx], dim=1)
        return self.classify_net(x)


model = IncomeWideDeepClassifier(embedding_sizes, crossing_sizes, 4)
```



Best Results:

Numeric Only: 78.94%
Embedded: 84.81%
Crossed: 85.09%